Lab2: BombLab

复旦大学
黄果 23301030064

一、实验环境与基本流程

本实验在 Linux x86-64 环境中完成，通过分析个人 bomb 的 x86-64 反汇编代码，恢复六个正式关卡及 Honor 关卡中 challenge 与正确口令之间的计算关系，并在 defuser.c 中实现对应的求解函数。分析时主要使用 objdump 导出关键函数汇编，结合 GDB 观察寄存器、栈和内存中的运行时数据，最后使用公开测试和额外 seed 验证实现是否正确。类似的二进制逆向分析通常需要从低层指令、调用关系和控制流结构中恢复程序语义[1]。

实验开始后，先领取并编译个人 bomb，再根据关卡逐步导出反汇编。每次修改 defuser.c 后重新编译，然后使用 defuser 生成口令并通过管道交给 bomb 检查。

在阅读汇编时重点关注函数调用前后的参数寄存器、cmp 附近的比较对象、循环与跳转结构，以及 eax、edi 等 32 位寄存器对应的运算宽度。解法中统一使用 uint32_t 保存关键中间结果。

二、测试结果与实现思路

1. 关卡测试结果

```
squaredog@DESKTOP-92P7DI3:~/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 98765432l: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
squaredog@DESKTOP-92P7DI3:~/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 98765432l: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
squaredog@DESKTOP-92P7DI3:~/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 98765432l: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
squaredog@DESKTOP-92P7DI3:~/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 98765432l: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
squaredog@DESKTOP-92P7DI3:~/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 98765432l: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
squaredog@DESKTOP-92P7DI3:~/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 98765432l: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 1 六个正式关卡公开测试结果

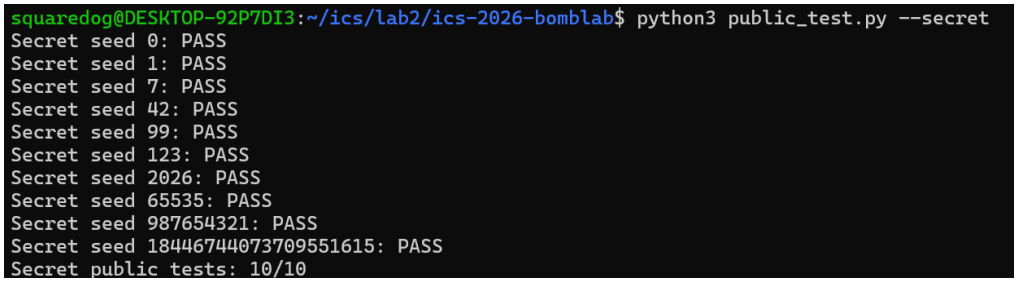


图 2 Honor 公开测试结果

2. 各关实现思路

下面记录每关用于恢复答案的关键汇编结构、最终计算关系和主要推理过程。

Phase1 位段提取与组合运算

本关只需要输出一个 32 位无符号整数。汇编先分别取得 challenge 的高 32 位、右移 16 位后的低 32 位和原始低 32 位，再通过异或、移位倍乘和加法构造两个中间量。关键汇编为对 challenge 执行 shr $0x20 和 shr $0x10，并使用 xor、lea 和 add 完成常数混合与倍乘。

```
1. shr $0x20, ...
2. shr $0x10, ...
3. xor $0x9e28, ...
4. lea (...,...,4), ...
5. lea (...,...,8), ...
6. xor $0x66e8, ...
7. add $0xd1db, ...
```

恢复出的关系为 high=(uint32_t)(challenge>>32)，mid=(uint32_t)(challenge>>16)，low=(uint32_t)challenge；随后 a=(mid^0x9e28)+4×high，b=(low+8×high)^0x66e8，最终答案为(a+0xd1db)^b。这里 mid 不是只截取 16 位，而是 challenge 右移 16 位后取低 32 位，因此不能写成(challenge>>16)&0xffff。

Phase2 数组、循环与递推关系

本关要求输出 6 个整数。第一个数由 challenge 中的一个 8 位字段产生，后面 5 个数通过循环递推得到。汇编中可以看到右移和 and 用于提取不同位段，循环体内通过 add、lea 和 sub 实现“前一项+循环下标→乘 5→减去当前 4 位字段”的过程。

```
1. shr / and    # 提取 8 位或 4 位字段
2. add    # 前一项与循环下标相加
3. lea (...,...,4), ...    # 计算 5*(prev+i)
4. sub    # 减去当前 4 位字段
```

计算关系为 values[0]=3×((challenge>>27)&0xff)。从 i=1 开始，shift 初值为 15，每轮取 t=(challenge>>shift)&0xf，然后计算 values[i]=5×(values[i-1]+i)-t，最后令 shift 增加 5。

Phase3 switch 分支与跳转表

本关先用 challenge 低 3 位得到 index，再从 challenge 构造统一的 key。随后通过间接跳转进入 8 个 case，因此反汇编中表现为典型的 switch 跳转表。输入需要两个整数，第一个为 index，第二个为对应分支计算出的 result。

```
1. and $0x7, ...    # index
2. shr $0x20, ...
3. shr $0x3, ...
4. xor ..., ...    # key
5. jmp *table(,...,8)    # 跳转表
```

其中 index=challenge&7，key=(uint32_t)(challenge>>32)^(uint32_t)(challenge>>3)。8 个分支依次为：case0：5×key+0xafff；case1：3×key+0xe084；case2：5×key+0xde6c；case3：5×key+0x13df3；case4：(key^0xb640)+(key>>1)；case5：(2×key)^0x7ee7；case6：(4×key)^0x74bd；case7：(key^0x8244)+(key>>2)。最终输出 index 和 result。

Phase4 递归调用与二分搜索

本关调用一个递归辅助函数，在区间[0,127]中按照二分搜索不断缩小范围。challenge 低 7 位作为 target，另一个由 challenge 构造出的 key 参与每一层返回值的混合。关键汇编体现为计算 mid、比较 target 与 mid，并根据小于或大于分别递归进入左右区间。

```
1. mid = low + (high-low)/2
2. cmp target, mid
3. je  base_case
4. jb  recurse_left
5. ja  recurse_right
```

恢复出 target=challenge&0x7fkey=(uint32_t)(challenge>>7)^(uint32_t)(challenge>>32)。当 target==mid 时返回 (mid+100)^key；当 target<mid 时先递归到 [low,mid-1]，再返回 (5×r)^(mid+key)^0x1a；当 target>mid 时递归到[mid+1,high]，返回(5×r)^(mid+key)^0x19。最终答案为 F(target,0,127)。由于返回值会逐层混入路径上的 mid，因此必须完整复现递归返回过程。

Phase5 结构体布局与链表遍历

本关的数据区包含 12 个节点，每个节点具有 value、tag 和 next 关系，同时存在 8 个不

同的链表起点。challenge 低 3 位选择起点，另外 2 位决定向后遍历的步数。反汇编中通过固定偏移读取节点字段，并沿 next 指针重复访问，因此可以还原出节点表和链表关系。

```
1. node_value =
{0x779,0x971,0x31c,0x555,0xa70,0x15c,0x079,0x817,0xb1b,0x444,0x24d,0x680}
2. node_tagi=
0x9dcf,0x1da7,0x0588,0xfaf3,0x68ad,0x32e1,0xbec7,0xe547,0x37e6,0x7fb3,0xa570,0xdce2}
3. node_next = {10,8,7,5,0,2,3,9,-1,1,6,4}
4. heads = {11,4,0,10,6,3,5,2}
```

head_index=challenge&7，steps=(challenge>>3)&3，起始 node=heads[head_index]。初始 result=(uint32_t)(challenge>>6)^(uint32_t)(challenge>>32)^node_tag[node]^0x2b83，再加上 node_value[node]。随后循环 steps 次，每轮沿 node_next 前进，并依次异或新节点的 tag、加上新节点的 value。最终输出 result 和遍历结束节点的 node_value。

Phase6 函数指针表与间接调用

本关从 challenge 中取得两个 3 位索引，并通过函数指针表连续执行两次变换。第一次变换作用于 key，结果与 0xc8dd 异或后作为第二次变换的输入，第二次变换完成后再加 0xe1e6 得到答案。

```
1. and $0x7, ...    # index1
2. shr $0x3 / and $0x7    # index2
3. call *table(,...,8)    # 第一次间接调用
4. xor $0xc8dd, ...
5. call *table(,...,8)    # 第二次间接调用
6. add $0xe1e6, ...
```

key=(uint32_t)(challenge>>6)^(uint32_t)(challenge>>32)，index1=challenge&7，index2=(challenge>>3)&7。函数表中 f0(x)=(x^0xd101)+2x；f1(x)=f2(x)=((x>>2)^x)+0x4f32；f3(x)=(8x+0xba1b)^(3x)；f4(x)=(x+0x91fe)^(x>>3)；f5(x)=f6(x)=(x>>7)|(x<<25)；f7(x)=((x<<7)|(x>>25))^0x9d37。先计算 r=f[index1](key)，再 r^=0xc8dd，随后 r=f[index2](r)，最终 answer=r+0xe1e6。

Honor secret_gate、secret_copy 与 secret_phase

secret_gate 首先把整行复制到 rbp-0xd0，然后在 rbp-0xb0 处检查常量 0x4e45504f。由于 x86-64 采用小端序，该常量对应 ASCII 字符串 OPEN，而 rbp-0xb0 相对缓冲区起点偏移 0x20，也就是第 33 个字符的位置。随后 secret_phase 接收 rbp-0xac，即偏移 0x24 处的地址，因此

三个整数应紧跟在 OPEN 之后。

```
1. 40273b: lea    -0xd0(%rbp),%r12
2. 402786: call   402700 <secret_copy>
3. 40278b: cmpl   $0x4e45504f,-0xb0(%rbp)
4. 40279d: lea    -0xac(%rbp),%rdi
5. 4027a4: call   402640 <secret_phase>
```

因此第七行格式应为 32 个非空填充字符+OPEN+三个无符号十进制整数。实现中使用 32 个字符 A 作为填充，即 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN%u %u %u。

secret_mix 只有几条指令，可以直接还原为 secret_mix(x)=((x>>7)^0xddc3)+(x^0xb1c1)。

```
1. 402624: mov    %edi,%eax
2. 402626: xor    $0xb1c1,%edi
3. 40262c: shr    $0x7,%eax
4. 40262f: xor    $0xddc3,%eax
5. 402634: add    %edi,%eax
```

在 secret_phase 中，第一个整数 a=(uint32_t)context&3。再令 s=high32(challenge)^low32(challenge)，t=high32(context)^low32(context)，x=s^t，mix=secret_mix(x)，u=(a×0xb1c1)^(uint32_t)(context>>16)，则第二个整数 b=mix^u，第三个整数 c=b^(t+a+0xddc3+(s>>7))。

3.非公开 seed 验证与运行结果

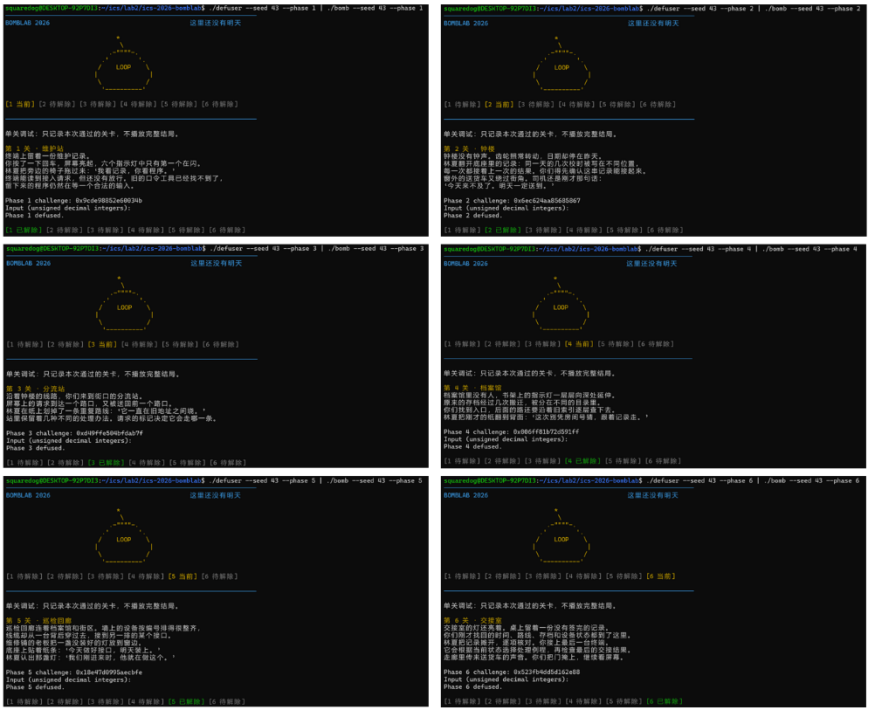


图 3 使用非公开 seed 43 验证全部已完成关卡

课程要求每个已完成关卡至少使用一次公开列表以外的 seed 验证。公开测试列表中没有 seed 43，因此可以统一使用 seed 43 检查六个正式关卡和 Honor。

4. 辅助工具与资料说明

本实验主要使用课程 Lab2 说明、CS:APP 第三章、GNU GDB 文档和 objdump 手册查阅 x86-64 调用约定、寄存器含义、反汇编和调试命令。实验过程中使用 ChatGPT 辅助解释部分 x86-64 指令语义、整理寄存器数据流、并辅助检查 32 位无符号运算。将反汇编关系改写为 C 表达式、测试验证与最终的实验报告撰写由本人独立完成。

三、实验总结

本次 BombLab 把 x86-64 汇编语言、函数调用约定、分支、循环、递归、结构体布局、链表以及函数指针等知识落实到了实际二进制程序分析中。相比直接阅读 C 代码，反汇编中变量名和类型信息已经消失，需要从寄存器传递、内存偏移和控制流中重新建立数据之间的关系。

六个正式关卡分别对应了位段运算、递推循环、switch 跳转表、递归返回、链表遍历和间接函数调用。Honor 进一步说明，仅分析被调用函数本身并不一定足够，调用者对输入字符串的预处理和内存布局同样可能决定最终答案。Honor 从 0/10 到 10/10 的调试过程也让我更加注意地址偏移、小端序和调用链之间的关系。

实际实现中，32 位运算宽度也是容易出错的地方。汇编中的 eax、edi 等寄存器对应 32 位运算，使用 uint32_t 能够自然保留模 2^32 回绕语义，避免有符号溢出或错误的 64 位扩展改变结果。在分析过程中，除静态阅读汇编外，还使用 GDB 设置断点、查看寄存器和内存中的运行时数据，以验证从汇编中推导出的计算关系。动态调试能够通过观察程序执行状态辅助定位和理解低层程序行为[2]。再和 public_test.py 批量测试相结合，可以较可靠地把汇编推导转化为对任意 challenge 均成立的求解函数。

四、建议

建议后续可以在实验说明中增加一个更短的“从函数调用者追踪参数来源”的示例，提醒初学者在辅助函数公式看似正确但测试仍失败时，不仅检查被调用函数，还要继续向上分析调用者对输入和缓冲区的处理。

另外，可以在 Honor 提示中适当强调字符串地址偏移与小端序的观察方法，而不直接给

出具体答案。这样既能保留探索性，也能减少因为只关注数值运算而忽略外层输入格式所造成的无效调试。

五、参考文献

[1] David Y, Alon U, Yahav E. Neural Reverse Engineering of Stripped Binaries Using Augmented Control Flow Graphs[J]. Proceedings of the ACM on Programming Languages, 2020, 4(OOPSLA): 225:1-225:28.
[2] Akgul T, Mooney V J III. Assembly Instruction Level Reverse Execution for Debugging[J]. ACM Transactions on Software Engineering and Methodology, 2004, 13(2): 149-198.